HOCHSCHULE
RAVENSBURG-WEINGARTEN
UNIVERSITY
OF APPLIED SCIENCES

RAVENSBURG-WEINGARTEN UNIVERSITY

Computer Architecture

Memory Hierarchy & Implementation

Presented by :
Rajveersinh Suratiya ()
Vivek Parmar (29943791)
Ramanjaneya Reddy Bhumireddy (29743760)

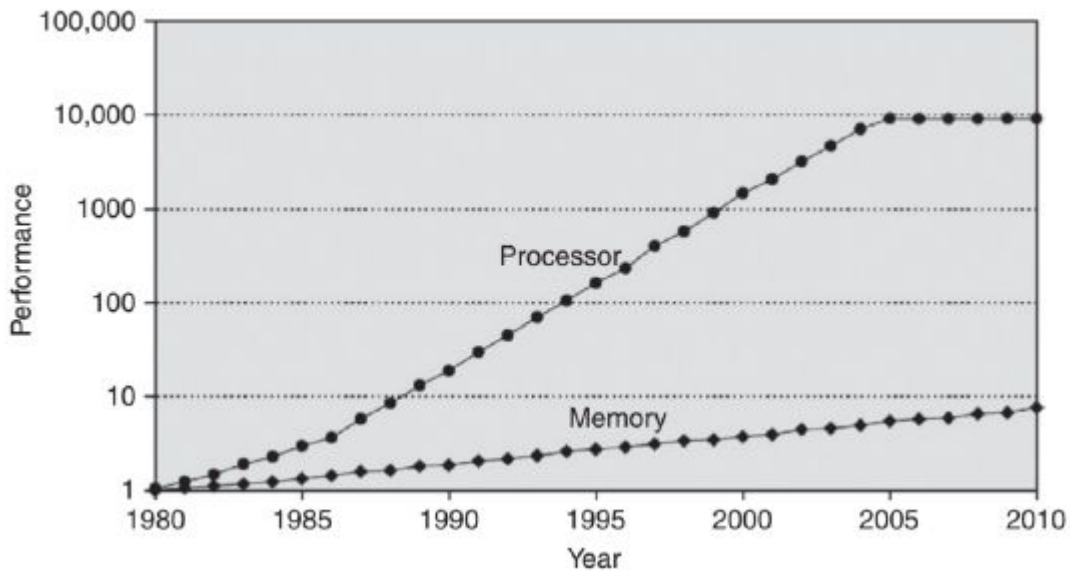
Under the Guidance of :
Prof. Dr. Ing. Andreas Siggelkow

Abstract

To overcome the performance gap between high-speed processors and slower memory, modern systems use a structured memory hierarchy. This presentation examines the roles of various memory levels—including registers, caches (L1), main memory (DRAM). It further highlights the design and functioning of a direct-mapped cache, supported by simulation results from a SystemVerilog implementation. And gives a overall view of the working of Memory Hierarchy.

The Memory Wall Problem

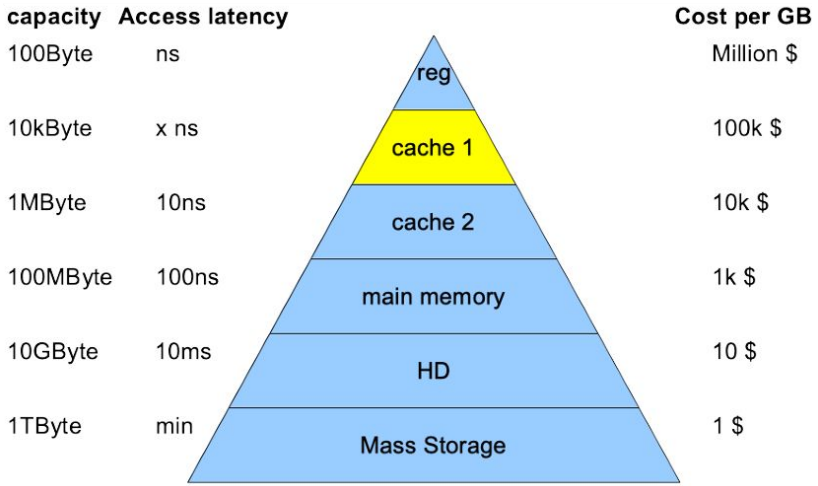
- CPU speeds have increased dramatically, while memory speeds have improved slowly. This growing speed gap is referred to as the **"Memory Wall."**
- Where the processor must frequently wait for slower memory operations, limiting performance.
  - CPUs outpace memory improvements (Moore's Law vs DRAM latency)
  - Leads to performance bottlenecks
  - Memory hierarchy helps mitigate this gap



Memory Hierarchy

- The memory hierarchy is a structured arrangement of storage systems designed to provide a balance between speed, cost, and size. As the distance from the CPU increases, the memory gets larger and cheaper—but also slower.
- This hierarchy helps minimize the latency of memory access and keeps the processor efficiently fed with instructions and data.

- Speed decreases as you move down the hierarchy
- Designed to reduce CPU idle time due to memory latency



Registers

- Registers are small, extremely fast memory elements inside the CPU used to hold data and instructions temporarily during execution.
- The topmost level of the memory hierarchy — even faster than cache, but very limited in number and size.
- Purpose is to Store operands, addresses, instructions during execution
- Type of Registers are General Purpose Registers, Program Counter, Instruction Register, Accumulator, Memory Address Register, Memory Data Register, Status/Flag Register, Stack Pointer.

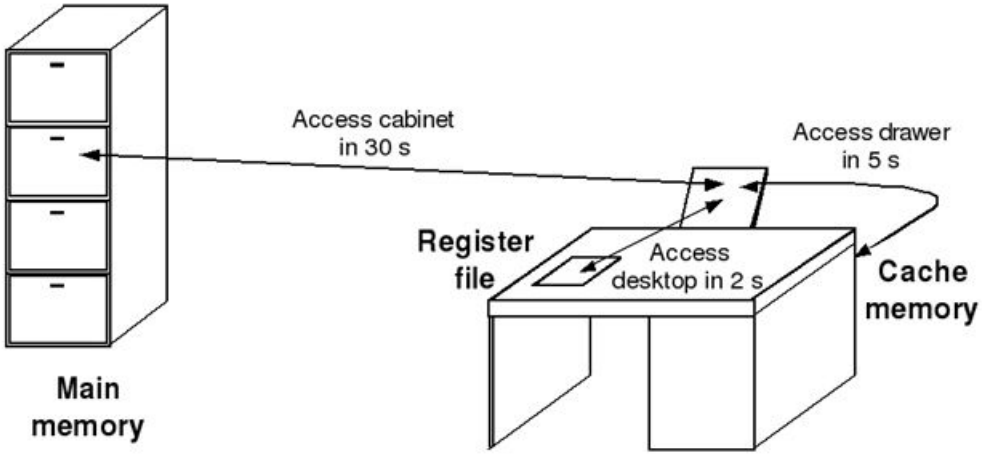
Cache Memory

- Cache memory is a small, fast memory placed close to the CPU to reduce the time it takes to access frequently used data. How data is placed in the cache is determined by mapping techniques.
- Cache mapping determines how memory blocks are assigned to cache lines.

There are three major types:

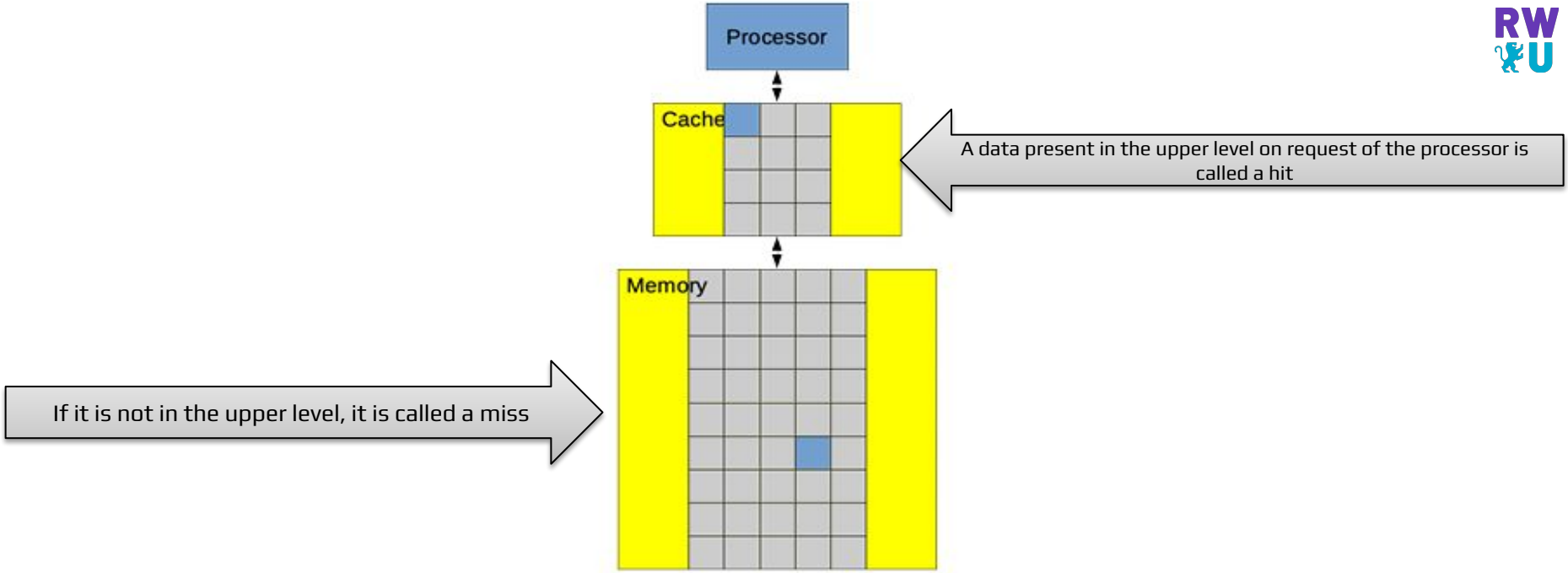
- Direct-Mapped Cache – Simple and fast; one memory block maps to one line
- Fully Associative Cache – Any block can go into any line; flexible but slow
- Set-Associative Cache – Compromise between the two; blocks go into a set of lines

Principle of Locality



- Temporal Locality : Recently accessed items are likely to be accessed again (e.g., loop variables).
- Spatial Locality : Items located near accessed addresses are likely to be accessed soon (e.g., arrays).

These principles justify the hierarchical layering of memory systems and guide cache architecture.



- **Block (or line)** : The minimum unit of information that can be either present or not present in a cache.
- **Hit rate** : The fraction of memory accesses found in a level of the memory hierarchy
- **Miss rate** : The fraction of memory accesses not found in a level of the memory hierarchy.
- **Hit time** : The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

Cache Operation in Memory Hierarchy

When the CPU reads or writes data, it first checks the cache by comparing address tags. If found (cache hit), it accesses data quickly. If not found (cache miss), data is fetched from main memory, which is slower. The cache must then store this new data, possibly evicting an old entry based on a replacement policy.

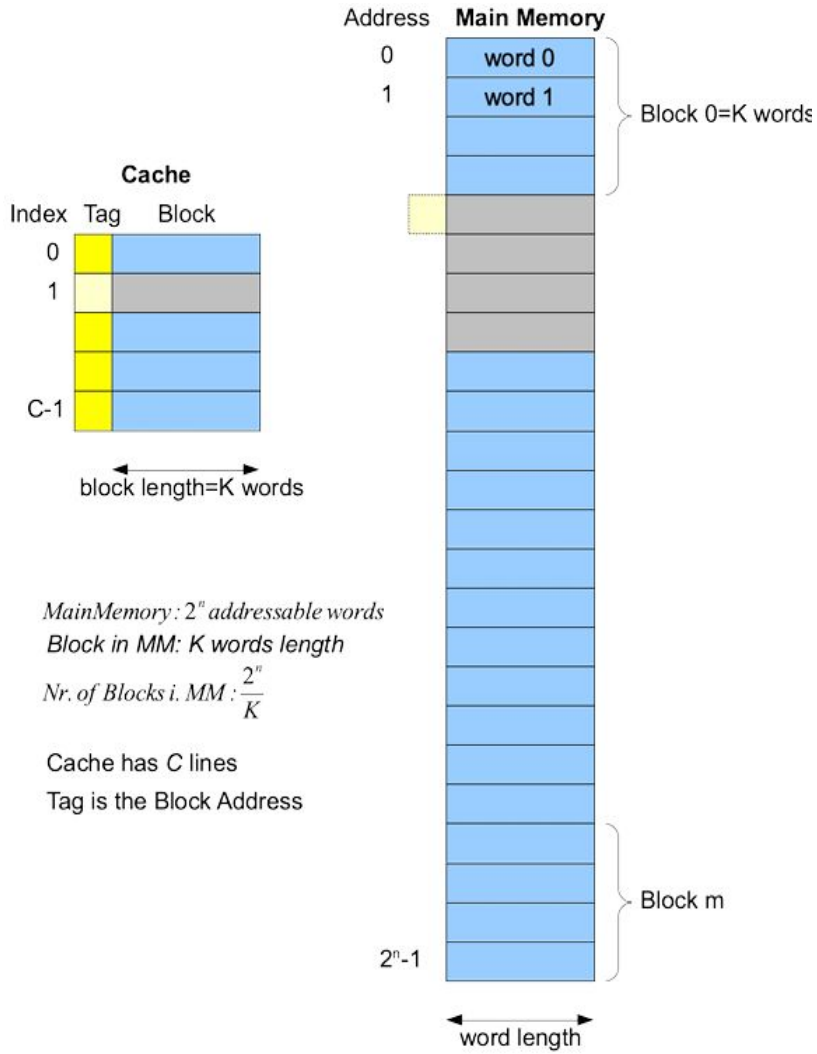
On a cache miss, the cache needs to make room for new data by evicting an existing entry based on a **replacement policy**. A common policy is **Least Recently Used (LRU)**, which removes the cache line that hasn't been accessed for the longest time.

When writing data, the cache follows a write policy:

- In a write-through cache, data written to the cache is immediately written to main memory.
- In a write-back cache, data is written only to the cache at first and marked as dirty. The data is written to main memory later, when the cache entry is replaced.
  - Write-through → write to cache + main memory at the same time
  - Write-back → write to cache first, update main memory later (on eviction)

Principle of a Cache

- The main memory is a 32 bit organization, one word is 32 bits wide
- The addresses here falsely increase by 1 and not by 4
- One cache block (also called line) has K words
- So, the number of bits needed for the displacement is Dlen = log2(K)
- The number of bits for the index is Ilen = log2(Cr)
- The number of bits for the tag is Tlen = 32 Ilen Dlen



Cache Memory Design Parameters

Cache memory is designed with several key parameters that impact its speed, efficiency, and cost. The right combination of these parameters helps balance performance and hardware complexity. Let's look at the important design choices in cache memory.

Key parameters:

- **Cache size** : The total storage capacity of the cache (in bytes or words).
  Larger cache holds more data but is more expensive and can be slower.

- **Block or cache-line size**: The size of the unit transferred between main memory and cache.
  Bigger blocks can improve hit rate but might bring in unnecessary data.

- **Placement policy**: Decides where the incoming data is stored in the cache.
  More flexible placement increases hardware cost and may or may not improve performance.

- **Replacement policy**: Chooses which cache block to overwrite when bringing in new data.
  Common strategies: random replacement, least recently used (LRU).

Cache Memory Design Parameters

- **Write policy**: Defines how updates to cache are handled.
  Write-through: Update both cache and main memory at the same time.
  Write-back: Update only cache; write to main memory when block is replaced. (Implemented in Project)

- **Hit**: Data is found in the cache.

- **Hit rate**: Fraction of memory accesses served by the cache.

- **Hit time**: Time taken to access data in cache (includes RAM access + checking for hit).

- **Miss**: Data is not in cache; must be fetched from main memory.

- **Miss rate**: 1 minus the hit rate (Miss rate = 1 - Hit rate).

- **Miss penalty**: Time to fetch data from main memory and deliver it to the processor.

Cache row structure

Data blocks	Tag	Index	Displacement	Valid bit

Each cache entry stores data blocks fetched from main memory. The memory address is split into parts to locate and verify data in the cache.

Memory address split into:

- **Data blocks** : Contain the actual data fetched from the main memory.
- **Tag** : Most significant bits; identifies if the data is the correct one.
- **Index** : elects the cache row (length = $\log_2(C_R)$).
- **Displacement (offset)** : selects the specific byte/block within the cache line (length = $\log_2(D_{blocks})$).
- **Valid bit**: Marks if the cache entry contains valid data.

Tag check: Ensures the cache row (found via index) matches the requested address.

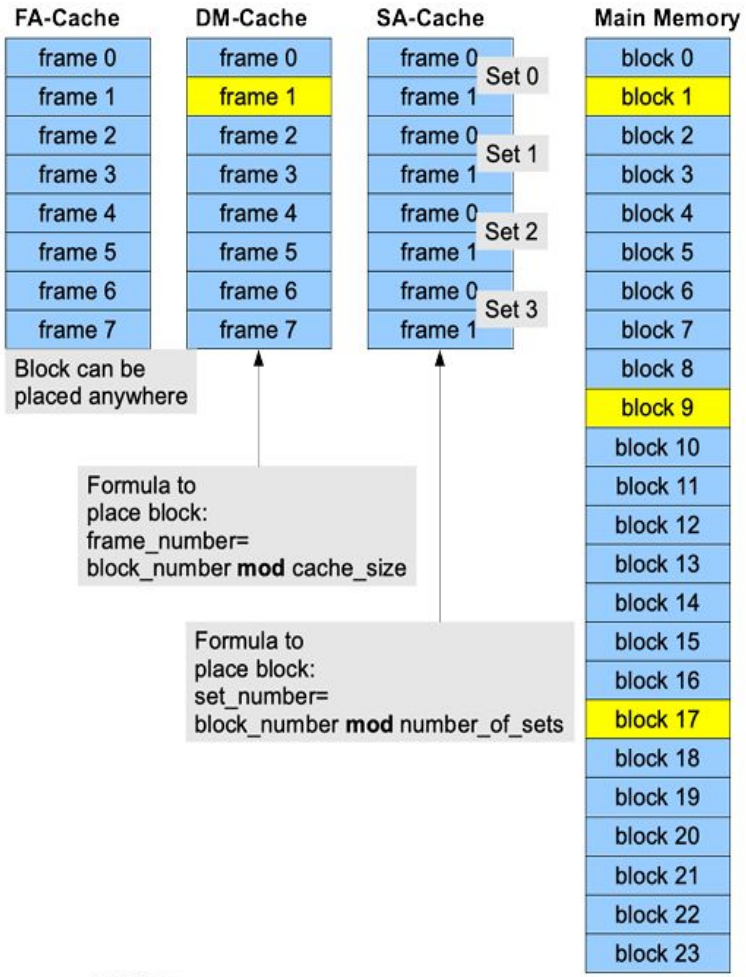
Tag length: = Address length – Index length – Displacement length ($T_{len} = Addr_{len} - I_{len} - D_{len}$)

(Addrlen: number of bits of the address)

Direct-Mapped Cache

If each entry in main memory can go in just one place in the cache, the cache is **direct mapped**.

- Each memory address corresponds to only a single cache location.
- But a single cache location can correspond to many memory locations



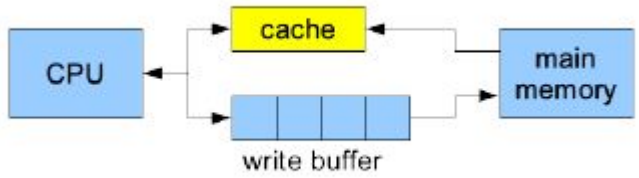
cache21.png

Cache Read And Write Operation

When the CPU needs data, it first looks in the cache. Reads happen more often than writes in the cache. There are two ways the cache handles writes — **write-through** and **write-back**.

Write-Through

- Data is written to both the cache and main memory at the same time.
- This keeps cache and memory always the same (consistent).
- Simple to use, good for systems with many processors.
- Can be slower, but a **write buffer** helps speed up writes.



Write-Back

- Data is written only to the cache at first.
- The data is written to memory later, when the cache block is replaced.
- Uses a **dirty bit** to remember which data changed.
- Faster writes and uses less memory bandwidth.
- Needs special rules (cache-coherence) to make sure all processors see the latest data.

Address Mapping in Direct-Mapped Cache

In a direct-mapped cache, each memory address maps to exactly one location in the cache. When the number of cache entries is a power of 2, this mapping is done using the lowest bits of the address.

For example, if the cache has 8 blocks ($2^3 = 8$), then the 3 lowest bits of the memory address are used as the cache index. This means multiple memory addresses can map to the same cache location, which is why we also need **tags** and **valid bits** to identify what data is currently stored in the cache.

Example:

Cache has 8 entries → use lowest 3 bits of address ($\log_2(8) = 3$ bits)

Addresses that map to the same cache location :

Memory address 00001_2 → Cache location 001_2

Memory address 01001_2 → Cache location 001_2

Memory address 10001_2 → Cache location 001_2

Address Mapping in Direct-Mapped Cache

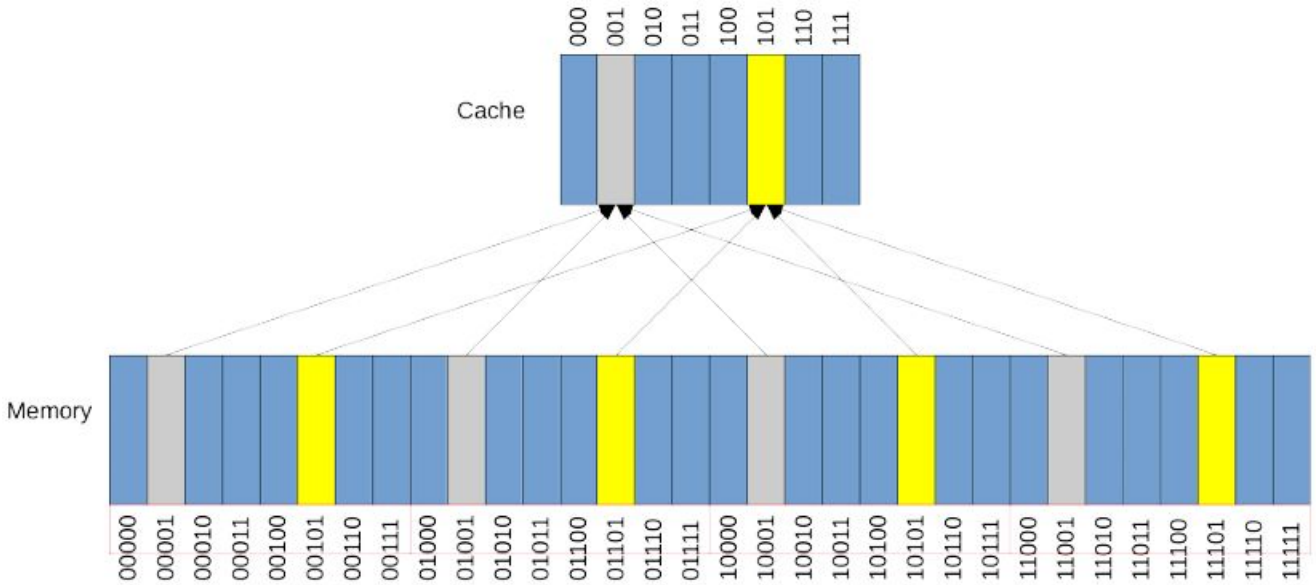
Memory address 11001_2 ⟶ Cache location 001_2

Memory address 00101_2 ⟶ Cache location 101_2

Memory address 01101_2 ⟶ Cache location 101_2

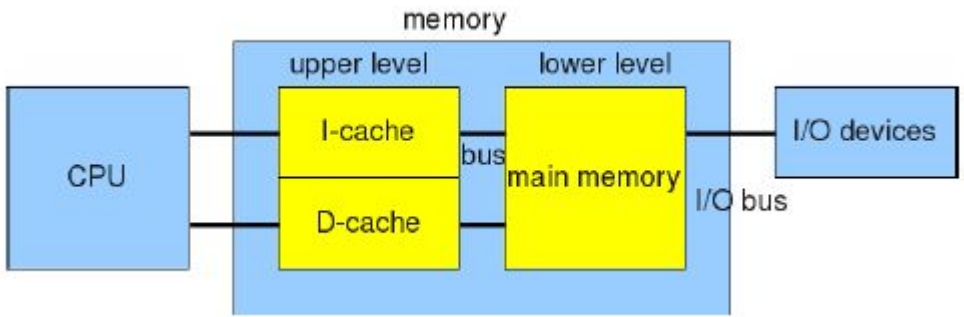
Memory address 10101_2 ⟶ Cache location 101_2

Memory address 11101_2 ⟶ Cache location 101_2



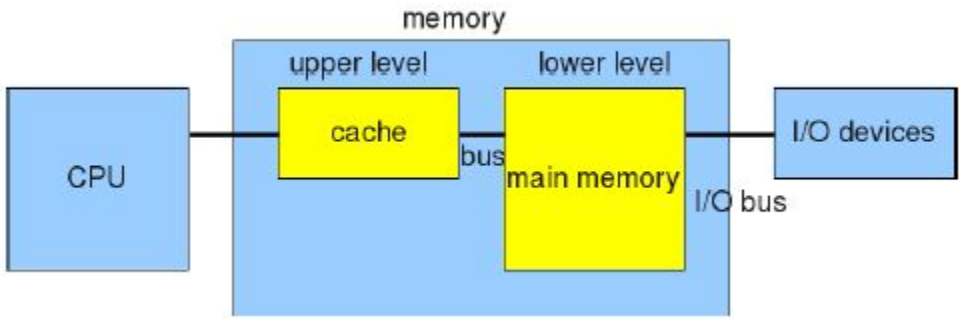
Memory Architectures
Harvard Architecture

- Separate memory for instructions and data
- Separate buses for data and instructions
- Can access instructions and data simultaneously
- Faster execution (simultaneous fetch of instructions and data)
- Avoids instruction/data conflicts (e.g., self-modifying code issues)
- Better suited for digital signal processing (DSP)
- More complex and expensive hardware (due to separate memories and buses)
- Less flexibility (data and program sizes are fixed separately)



Memory Architecture
Princeton (Von Neumann) Architecture

- Single shared memory for both
- Only one access (data or instruction) at a time
- Simpler and cheaper to design
- More flexible memory usage (single memory pool)
- Slower execution (instruction fetch and data access can't happen simultaneously)
- Common in general-purpose computing (e.g., PCs)



Project Specifications Implemented

Cache Type = Direct-Mapped Cache

Cache Size = 1 KB (1024 bytes)

Block Size = 16 bytes

Number of Cache Lines = 64

Address Width = 32 bits

Tag Width = 22 bits (in current config)

Index Width = 6 bits

Offset Width = 4 bits

Replacement Policy = None (Direct-Mapped)

Write Policy = Write-Back, Write-Allocate

Dirty Bit Support = Yes

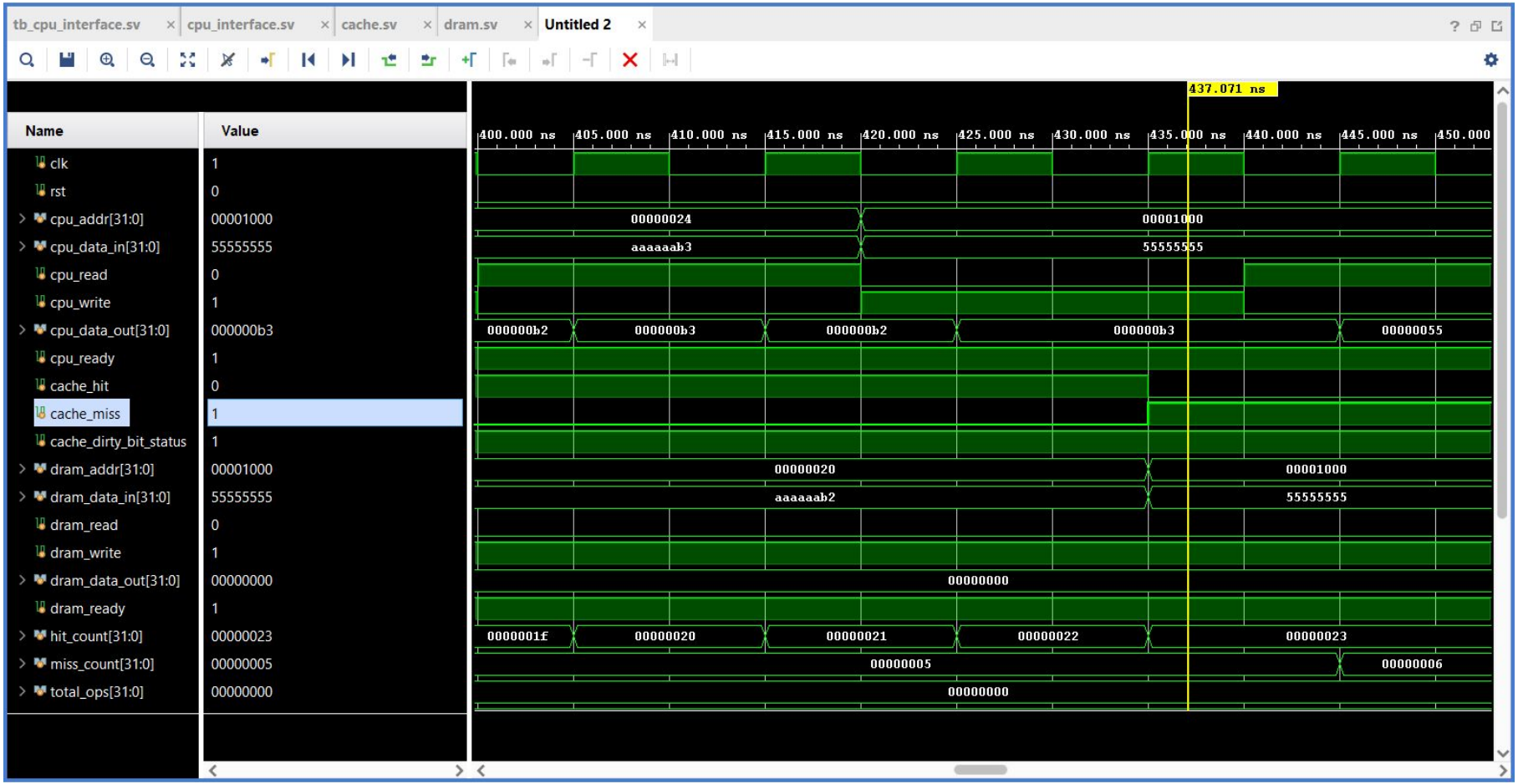
DRAM Size = 64 KB (65536 bytes)

Access Latency = clock cycle

Address Width = 32 bits

DRAM Access Granularity = Word-aligned (addr >> 4)

Output Waveform



Working of the System

- The memory hierarchy system operates by handling read and write requests from the CPU through a structured data flow between the CPU, cache, and DRAM. When the CPU initiates a memory operation, the cpu_interface forwards the request to the cache.

- In the case of a write operation, the cache checks if the corresponding block is present (cache hit) using the tag and valid bit. If it is a hit, data is written directly to the cache and the dirty bit is set. If it is a miss, the data is written to DRAM, and the cache is updated with the new block and marked as valid and dirty.

- For a read operation, the cache again checks for a hit. On a cache hit, data is directly sent to the CPU from the cache. On a cache miss, data is fetched from DRAM, stored in the cache, and then forwarded to the CPU.

Future scope of Improvements

- Upgrade to Set Associative or Fully Associative Cache.
- Add replacement policies like LRU, FIFO, Random.
- Model Multi Cycle latency.
- Introduce L2, L3 Cache (Multi Level Cache Hierarchy).

Conclusion

- This project successfully demonstrates the fundamental concepts of a memory hierarchy system by implementing a CPU interface, a direct-mapped cache, and a DRAM module using SystemVerilog.

- Through simulated read and write operations, it showcases how a cache layer can significantly reduce memory access latency by minimizing direct communication with slower main memory.

- The use of valid and dirty bits allowed the implementation of a write-back and write-allocate policy, highlighting realistic cache behaviors such as hit/miss detection and deferred DRAM updates.

- The testbench effectively captured performance metrics like hit and miss rates, offering insights into the efficiency of cache operations.

Thank YOU !